

## Resend Take-Home: Customer Tickets - Ivan Dans

How I approached this: I wrote the internal notes first, separating what the ticket states from what I am inferring and what I still need. The customer response is written from those notes. Every ticket ends with one upstream prevention idea, because a ticket reaching a human is a signal that something in the try block needs to change.

Labels use Resend's own taxonomy (Reliability, Usability, Functional, Deliverability, Abuse, Success Outreach) plus the product area.

### Priority order:

| Priority | Ticket   | Label                           | Why this position                                                                                                                                                                                                |
|----------|----------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1        | RES-7921 | Reliability / Sending           | Thousands of time-sensitive auth emails failed over four hours. Treated as a potential incident until logs say otherwise. <b>Escalated below.</b>                                                                |
| 2        | RES-1348 | Reliability / API authorization | Active, ongoing failure with unknown cause. Possibly a complete sending block. Checked in the same first sweep as #1: this one is still bleeding, #1 is aftermath at larger scale, and the two may share a cause |
| 3        | RES-3485 | Usability / Rate limits         | Active notification failures, but the error is understood and has an immediate mitigation                                                                                                                        |
| 4        | RES-2196 | Deliverability / Gmail          | Material business impact, but mail is flowing. Needs evidence before any diagnosis                                                                                                                               |
| 5        | RES-1927 | Usability / Domains             | Customer blocked during setup. Quick, well-documented fix                                                                                                                                                        |
| 6        | RES-5842 | Usability / Receiving           | Implementation question, nothing failing                                                                                                                                                                         |
| 7        | RES-2984 | Usability / Sending             | Ambiguous how-to, no active failure. Fast to answer, so it will not wait long despite the rank                                                                                                                   |

Priority is about sequencing under constraint, not importance. 5 through 7 are minutes each; if the queue allows, they get cleared quickly rather than aging behind the investigations.

**Ticket RES-7921. "My emails suddenly stopped sending last night for 4 hours and thousands of magic links didn't send. What happened? This is unacceptable."**

**Label:** Reliability / Sending. **Priority:** 1.

**Note explaining my thinking:**

**Facts from the ticket:** a roughly four-hour window last night, high volume, magic links, so time-sensitive authentication mail whose failure blocks their end users from logging in. The window appears to be over.

**Not yet known:** whether requests failed at the API (4xx/5xx responses in their logs), were accepted and then failed downstream (accepted at our edge, no delivery events), or never arrived at all (their side: deploy, queue, credentials). These are three different incidents with three different owners.

**Process:**

1. Check the status page and internal incident channel for last night before replying. Never let the customer know about our outage from us asking them questions.
2. Pull the account's sending volume graph for the window. A cliff to zero with normal volume either side is the fastest scope signal.
3. Split accepted-vs-delivered: if the API returned IDs, check the event timeline on a sample. If the API returned errors, get the status codes from their logs.
4. Check whether other accounts show the same window. Multi-account means incident process, status page, and this stops being a ticket.
5. **Do not advise a blind resend.** Magic links expire; replaying a four-hour-old batch sends thousands of dead links and creates a second incident. Any replay must regenerate links, and that is their application logic, not ours.

**Judgment call:** this is ranked above the active 403s because of end-user scale, recurrence risk, and because its diagnosis may explain #2. Both are checked in the same first fifteen minutes.

**Draft response to the customer:**

Hi, I understand how serious this is. Four hours of failed magic links means your users could not log in, and you need to know what happened and whether it can happen again. I am treating this as a potential incident and investigating now, not just answering a ticket.

What I am doing on our side: pulling your account's sending activity for last night to establish the exact window, and checking each failed send to see whether requests were rejected at our API, accepted but not delivered, or never reached us. Those point to different causes and I do not want to guess.

Two things that would speed this up: the approximate time window in UTC, and two or three email IDs or API responses from your logs during the failure, if you have them.

One caution in the meantime: please do not bulk-resend the failed batch as-is. Magic links from last night will have expired, and a resend would deliver thousands of dead links. If a resend is needed, the links should be regenerated first.

I will update you within the hour with what I have found, even if the answer is not complete yet.

**How I would stop this coming back:**

An account-level alert when a customer's send volume drops to zero against their own baseline. We should be telling customers about a four-hour cliff, not hearing about it from them the next morning.

**Ticket RES-1348. "I'm seeing a ton of 403 errors on my account. How do I fix that?"**

**Label:** Reliability / API authorization. **Priority:** 2.

**Note explaining my thinking:**

**Facts:** ongoing 403s, volume implied high. Nothing else. **A 403 means the request was refused, and the status alone does not say why.** The exact error name and message determine the case, and the possibilities include the key itself being bad: Resend returns 403 for inactive and suspended keys, not just for permission problems.

**Documented 403 causes, each with a different fix:** an inactive or suspended API key, a key missing the required permission scope, the testing restriction (sending to any address other than the account owner's before a domain is verified), and sending from an unverified domain. Hand-rolled HTTP clients can also be refused at the edge before reaching the API. Nothing here can be told apart from the status code alone, which is why the error body is the first ask.

**Process:** get the exact JSON error body, never the key. The body's name field distinguishes every cause above. Check the account's domain list and verification status in parallel, since that answers half of these without waiting on the customer.

**Draft response to the customer:**

Happy to dig into this. A 403 from our API means the request was refused, and there are several distinct causes with different fixes, including some where the key itself is the problem. So rather than have you try things, let us identify which one this is.

Could you send me the full JSON error body from one of the failing requests? It looks like { "statusCode": 403, "name": "...", "message": "..." }. The name field tells us exactly which case we are in. Please do not include your API key; I never need it.

While you grab that, I am checking your account from this side. The usual causes are an API key that is inactive or suspended, a key without the permission the request needs, sending to addresses other than your own before a domain is verified, or sending from a domain that is not verified yet. All of them are quick fixes once we know which one it is.

If the errors started suddenly after working fine, tell me roughly when. That timing matters.

**How I would stop this coming back:**

Every 403 body should carry a docs URL naming its specific cause and fix. The error already knows why it refused; telling the customer removes the ticket.

**Ticket RES-3485. Rate limit error: "Too many requests. You can only make 2 requests per second."**

**Label:** Usability / Rate limits. **Priority:** 3.

**Note explaining my thinking:**

**Facts:** the error body itself states this account's limit is 2 requests per second. Trust the account's own headers over any remembered default; limits vary per account and change over time.

**Key semantics:** a request rejected with 429 was not accepted and not sent, so retrying it cannot duplicate an email. The limit is also **team-wide across all API keys**, so a second service on the same account counts against the same budget, which is a classic source of "we did not change anything" rate-limit surprises. For anything retried automatically, an idempotency key per notification makes the retry safe even when the failure is ambiguous.

**Process:** confirm the account's configured limit internally, check whether their volume justifies an increase, and find out what their sending pattern is (burst on an event, or steady).

**Draft response to the customer:**

You are hitting the rate limit on your account, which is 2 requests per second, and the requests that get the 429 are not being sent, which is why those users never receive the notification.

Two fixes, one immediate and one structural.

Immediate: instead of firing requests as events happen, put sends into a small queue that dispatches within your limit, and on any 429, wait the number of seconds in the `retry-after` response header before retrying. A rejected request was never accepted, so retrying it is safe and will not double-send.

Structural: if your legitimate volume is simply above 2 per second, we can look at raising the limit on your account. Tell me your typical and peak sends per minute and I will take it from there. Two more things worth knowing: the limit is shared across every API key on your team, so other services sending on the same account count against it, and the `ratelimit-limit`, `ratelimit-remaining` and `ratelimit-reset` headers are on every response, so your code

can pace itself against them rather than hard-coding a number. If you automate retries, attach an idempotency key to each notification so a retry can never double-send.

**How I would stop this coming back:**

An opt-in retry helper in the official SDKs: bounded attempts, honours retry-after, requires an idempotency key. Opt-in rather than default, because silent automatic retries change delivery semantics under the customer's feet. Most customers hit this once and hand-build the same queue; the helper removes that work without changing behaviour for anyone who did not ask.

**Ticket RES-2196. "My emails are going to the spam folder at Gmail. What can I do to stop this?"**

**Label:** Deliverability / Gmail. **Priority:** 4.

**Note explaining my thinking:**

**Facts:** spam placement at Gmail specifically. Nothing about volume, mail type, domain age, or authentication state.

**Discipline:** spam placement is multi-causal. Authentication, domain age and reputation, content and link alignment, list quality, engagement, volume patterns. One test send proves nothing in either direction, and nobody can honestly promise inbox placement. Diagnose from evidence: a Resend email ID plus the raw Authentication-Results header from a received message.

**Process:** check the domain's verification and DMARC state on our side, check spam-rate signals if available, then work the checklist in order: authentication first because it is binary, then link and domain coherence, then list and engagement questions.

**Draft response to the customer:**

Spam placement is diagnosable, but I want to work from evidence rather than a generic checklist, because the fix depends on which signal Gmail is reacting to.

Two things to send me: the email ID of a message that landed in spam, and the raw headers of that message as received (in Gmail: three dots, Show original). The Authentication-Results line tells us whether SPF, DKIM and DMARC are passing and aligned, which is the first fork in the road.

Meanwhile, the factors that most often matter, in order: authentication passing and aligned; links in the message pointing at your own sending domain rather than third-party or shortened URLs; whether the domain is new and warming up, since a new domain sending high volume immediately is a classic filter trigger; and for bulk mail to Gmail, one-click unsubscribe and keeping the spam-complaint rate low, which you can watch in Google Postmaster Tools.

To be straight with you: nobody can guarantee inbox placement, and I will not pretend otherwise. What we can do is eliminate every negative signal that is in your control, working from evidence rather than guesswork, and that is where these investigations should start.

**How I would stop this coming back:**

Resend already ships per-message Deliverability Insights covering authentication, link domains, DMARC and related checks, so the gap is not the feature, it is the route to it. Link Insights directly from spam-related support flows and from the agent's answers to this class of ticket, and consider surfacing raw Authentication-Results there, so the customer starts from the evidence instead of a ticket.

**Ticket RES-1927. "I'm not sure how to add the TXT record at Vercel. Can you tell me how?"**

**Label:** Usability / Domains. **Priority:** 5.

**Note explaining my thinking:**

**Facts:** mid domain-verification, DNS hosted at Vercel. The usual failure here is pasting the full hostname into a field that auto-appends the domain, creating `send.example.com.example.com`.

**Process:** confirm Vercel is actually authoritative for the domain (customers sometimes bought at Vercel but host DNS elsewhere, or vice versa). Then either the automatic flow or the manual steps.

**Draft response to the customer:**

Sure. Two ways to do this, and the first is faster.

Automatic: on your domain's page in Resend there is an "Auto Configure" option for Vercel that sets the records up for you. If your DNS is hosted at Vercel, that is the one-click path.

Manual: in Vercel, go to your team's Domains section (not the project), select the domain, and open DNS Records. Add a record with type TXT. In the Name field, enter only the part before your domain: if Resend shows `resend._domainkey.send.yourdomain.com`, the Name is `resend._domainkey.send`. Vercel appends your domain automatically, and pasting the full hostname is the most common reason verification fails, because it creates a doubled name. Paste the value exactly as Resend shows it, save, then click Verify back in Resend. It usually confirms within a few minutes.

If it does not verify within half an hour, send me a screenshot of the record as saved in Vercel and I will spot the difference.

**How I would stop this coming back:**

Provider-aware instructions on the domain page: detect the nameservers and show the exact field values for that provider, with the auto-append warning inline. This one question exists for every DNS host.

**Ticket RES-5842. "I need to be able to receive emails from Resend. How do I do that?"**

**Label:** Usability / Receiving. **Priority:** 6.

**Note explaining my thinking:**

**Facts:** ambiguous phrasing. Most likely they want to receive inbound mail at their domain through Resend. Could also mean receiving Resend's own notifications. Answer the likely reading and leave a door open for the other.

**Key concepts to convey:** every account already has a managed receiving address at `<id>.resend.app`, and received messages are viewable in the dashboard's Receiving tab with no webhook needed, so the fastest test costs nothing. For applications, the `email.received` webhook carries the email ID and metadata, not the full body: the content comes from the Receiving API using that ID, and attachments come via temporary download URLs. Custom domains work by adding an MX record; if their root domain already handles normal mail, a receiving subdomain avoids touching the existing MX.

**Draft response to the customer:**

Yes, and the fastest way to try it costs nothing to set up. Your account already has a managed receiving address: open Emails, select the Receiving tab, and copy your `<id>.resend.app` address. Anything sent to any username at that address shows up right there in the dashboard.

If your application needs to react to incoming mail, subscribe a webhook to the `email.received` event. One thing worth knowing up front: that event carries the email ID and metadata rather than the full message, so your endpoint uses the ID to fetch the body and headers through the Receiving API, and attachments come as temporary download URLs.

To receive at your own domain instead, you add the MX record shown in the receiving section. If your domain already handles regular mail, point the MX at a subdomain like `inbound.yourdomain.com` rather than the root, so your existing mailboxes are untouched.

If what you are after is a regular inbox where people read and reply, Resend is not that, and you would pair it with a mailbox provider. Tell me a little about what you are building and I can confirm which shape fits.

**How I would stop this coming back:**

The phrase "receive emails" carries both meanings, and this ticket will recur. A short "is Resend an inbox?" entry in the receiving docs, stating plainly what it is and is not, would let the docs and the support agent answer it.

**Ticket RES-2984. "How do i create an email?"**

**Label:** Usability / Sending. **Priority:** 7.

**Note explaining my thinking:**

**Facts:** almost none. "Create an email" could mean send one via the API, design a template, send a broadcast to an audience, or create a sender address like support@theirdomain.com. The last one has a pleasant answer: Resend does not require registering individual senders, so once the domain is verified they can send from any address on it. The reading changes the answer entirely, and guessing wrong wastes their time and ours.

**Approach:** one clarifying question, but never a bare one. Give the fastest path for each likely reading so they can self-serve immediately, whichever they meant.

**Draft response to the customer:**

Happy to help. Quick question so I point you at the right thing, because "create an email" can mean a few things here. Which is closest?

Sending an email from your code: the quickstart takes you from an API key to a sent email in a few minutes, in whatever language you are using.

Designing how an email looks: have a look at templates, or React Email if your emails live in code.

Sending one message to a list of subscribers: that is Broadcasts, created in the dashboard, no code needed.

Setting up an address to send from, like support@yourdomain.com: no registration needed. Once your domain is verified, you can send from any address on it.

Tell me which of those you are after, and what you are building generally, and I will give you exact steps rather than links.

**How I would stop this coming back:**

This is the clearest possible candidate for agent-first resolution: a well-scoped clarifying flow answers it without a human. If tickets this generic are reaching people, the agent's intake or the docs entry point needs the same disambiguation this reply does.



## Escalation message to the engineering team, for RES-7921

**To:** engineering on-call **From:** support (Ivan) **Severity:** High. Proposing incident classification pending scope check. **Subject:** ~4h sending failure window last night, account [ACCT-ID], thousands of failed auth emails. Scope unknown.

**Summary.** Customer reports sending stopped for roughly four hours last night (window per customer: approx. 22:00 to 02:00 UTC, unconfirmed; details marked [ASSUMED] pending their logs). Thousands of reported missing auth emails. The mail is login magic links, so each missing message blocked an end user from signing in. Sending appears recovered now.

### The bug, as currently evidenced:

[ASSUMED FOR EXERCISE] During the affected window, sample POST /emails requests returned 200 with email IDs, but those messages emitted no email.sent or email.failed events and sat with no terminal state for approximately four hours. The same payload succeeded before and after the window.

- **Expected:** an accepted request enters the sending pipeline and emits its lifecycle events.
- **Actual:** requests were accepted and then stranded between acceptance and dispatch, with no events and no error surfaced to the customer.
- **Bug:** accepted messages stalled before dispatch. Note the distinction: a bounce or recipient-side delay after dispatch would not be a platform bug. Accepted-but-never-dispatched, with no failure event, is.

### What support has verified so far:

- Account sending graph shows [ASSUMED: a drop to zero inside the reported window, normal volume either side]
- Status page check for the window: [PENDING], the report says "last night" with no date
- Sample email IDs from the customer: [PENDING]

**Reproduction and verification path:** pull API request logs for [ACCT-ID] for the window, split by response status; for a sample of accepted IDs, check event timelines for missing delivery events; then run the same query unscoped by account to establish whether this is single-tenant or platform-wide.

### Why urgent despite being recovered:

1. Cause unknown means recurrence risk is unknown, including tonight at the same time.
2. If any other accounts show the same window, this is an unreported platform incident and needs a status page entry and a postmortem, not a ticket.

3. The customer is asking "what happened" and deserves a real answer. I am holding them with an investigation update and have committed to hourly updates.
4. Replay is unsafe from the customer side (expired magic links), so "just resend" is not a mitigation and they know it.

**Asks:** confirm scope (single account or wider) within the hour if possible; root cause when known; and a decision on retroactive status page entry if scope is more than this account. I will own all customer communication.

*Some details in this escalation are placeholders marked [ASSUMED] or [PENDING], per the exercise instructions, since the ticket contains no logs, account ID, or timestamps.*